HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Prof. Yann Thoma

Formal verification
Verification of an AXI module
CERN training - November 2026

Context

We are interested in verifying a module accessible through an AXI-lite interface.
Its entity is the following :

```
entity axi4lite_slave is
    generic (
        -- Width of S_AXI data bus
        AXI_DATA_WIDTH  : integer    := 32;  -- 32 or 64 bits
        -- Width of S_AXI address bus
        AXI_ADDR_WIDTH  : integer    := 12
    );
    port (
        clk_i           : in  std_logic;
        reset_i         : in  std_logic;
        -- AXI4-Lite
        axi_awaddr_i    : in  std_logic_vector(AXI_ADDR_WIDTH-1 downto 0);

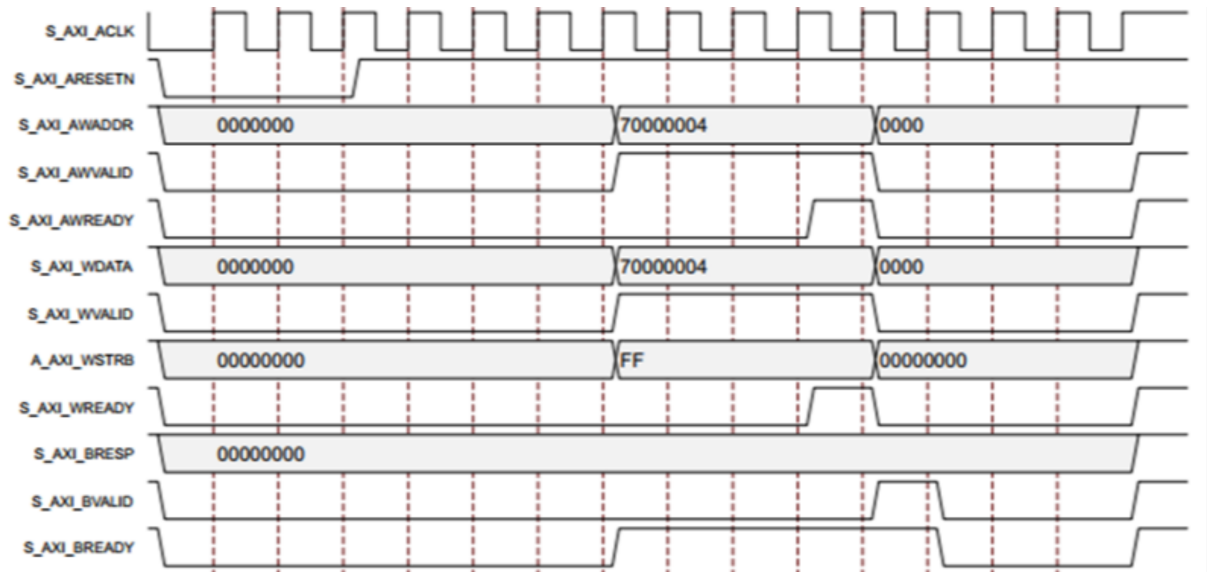
        axi_awvalid_i   : in  std_logic;
        axi_awready_o   : out std_logic;
        axi_wdata_i     : in  std_logic_vector(AXI_DATA_WIDTH-1 downto 0);
        axi_wstrb_i     : in std_logic_vector((AXI_DATA_WIDTH/8)-1 downto 0);
        axi_wvalid_i    : in  std_logic;
        axi_wready_o    : out std_logic;
        axi_bresp_o     : out std_logic_vector(1 downto 0);
        axi_bvalid_o    : out std_logic;
        axi_bready_i    : in  std_logic;
        axi_araddr_i    : in  std_logic_vector(AXI_ADDR_WIDTH-1 downto 0);
        axi_arvalid_i   : in  std_logic;
        axi_arready_o   : out std_logic;
        axi_rdata_o     : out std_logic_vector(AXI_DATA_WIDTH-1 downto 0);
        axi_rresp_o     : out std_logic_vector(1 downto 0);
        axi_rvalid_o    : out std_logic;
        axi_rready_i    : in  std_logic;
        -- User input-output
        input_reg_A_i   : in  std_logic_vector(31 downto 0);
        input_reg_B_i   : in  std_logic_vector(31 downto 0);

        output_reg_A_o  : out std_logic_vector(31 downto 0);
        output_reg_B_o  : out std_logic_vector(31 downto 0);
        output_reg_C_o  : out std_logic_vector(31 downto 0)
    );
end entity axi4lite_slave;
```

The AXI address is a byte address. In the formal configuration, `AXI_DATA_WIDTH` is 32 bits, so each word occupies 4 bytes and the two least-significant address bits select a byte within a word (the register map uses word-aligned addresses). The table lists byte offsets in hexadecimal ; for example, byte address `0x14` selects word index 5.
The module embeds different functionalities, corresponding to the following mapping :

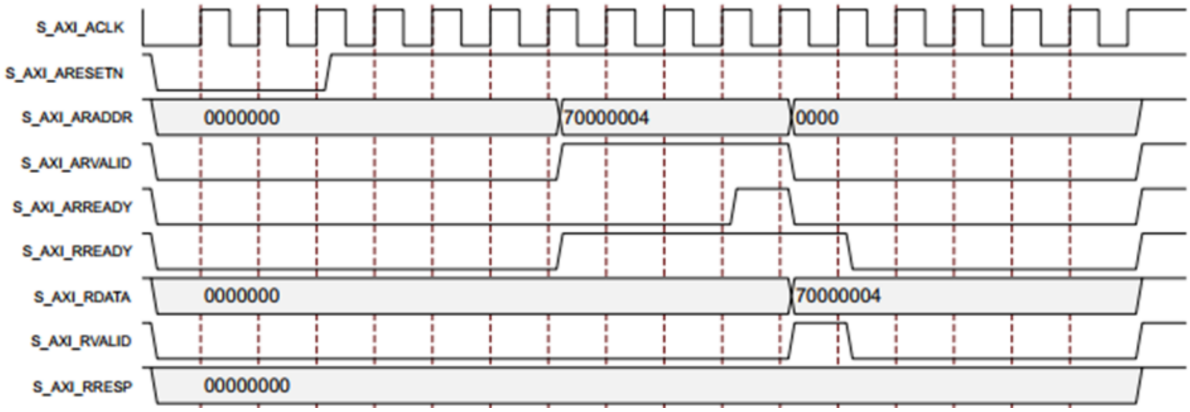
Byte offset	Direction	Description
0x00	Rd	Constant value : x"BADB100D"
0x04	Rd/Wr	Simple register
0x08	Rd	Reads the value of `input_reg_A` sampled when the read address is accepted
0x10	Rd	Reads the value of `input_reg_B` sampled when the read address is accepted
0x14	Wr	Sets the `output_reg_A` value
0x20	Wr	Sets the `output_reg_B` value
0x24	Wr	Sets the `output_reg_C` value
0x28	Wr/Rd	Sets or reads the counter value
0x2C	Wr	Writing at this address increments the counter

For reads at byte offsets 0x08 and 0x10, a read address is accepted on a rising edge where `axi_arvalid_i` and `axi_arready_o` are both high. At that edge, the corresponding input is sampled into an internal register. On the following rising edge, the sampled value is copied to `axi_rdata_o` and `axi_rvalid_o` is asserted. Thus the response returns the input value sampled at the address-acceptance edge, even if the input changes afterward. The read data remains stable while the response is valid and waiting for `axi_rready_i`.
An AXI transfer takes place on a rising clock edge when its `VALID` and `READY` signals are both high. The source must keep `VALID` asserted and hold the associated payload stable until that handshake occurs. First verify this rule for the write-address channel (`axi_awvalid_i/axi_awready_o` with `axi_awaddr_i`), write-data channel (`axi_wvalid_i/axi_wready_o` with `axi_wdata_i` and `axi_wstrb_i`), and read-address channel (`axi_arvalid_i/axi_arready_o` with `axi_araddr_i`). Then check that each response channel holds `VALID` and its payload stable while the receiver is not ready : `axi_bvalid_o/axi_bready_i` with `axi_bresp_o`, and `axi_rvalid_o/axi_rready_i` with `axi_rdata_o` and `axi_rresp_o`. Treat the address and data write channels as independent handshakes ; do not assume they are accepted on the same cycle. After checking these protocol properties, verify the register-map behavior.
A write transaction corresponds to the following waveform [1] :



A read transaction corresponds to the following waveform :

1. The waveform have been taken from https ://www.realdigital.org/doc/a9fee931f7a172423e1ba73f66ca4081.



Your job is to verify this module using formal verification. Start with the AXI protocol properties above, then add assertions for the module's register behavior.